

M01_123450048

Starter Notebook - Modul 1: Fondasi Jaringan Saraf, FNN, Aktivasi, dan Loss

Cell 1 - markdown

Modul 1: Fondasi Jaringan Saraf, FNN, Aktivasi, dan Loss

****Nama:**** Mahasiswa 123450048

****NIM:**** 123450048

****Kelas:**** -

****Tanggal:**** 2026-09-22

****Berkas pengumpulan:**** `M01\_123450048.ipynb`, `M01\_123450048.pdf`, dan `M01\_123450048\_metrics.csv`.

> Seluruh kode, eksperimen, grafik, dan analisis merupakan pekerjaan individual. Beri atribusi pada kode yang diadaptasi dari sumber lain.

Cell 2 - markdown

Petunjuk

1. Lengkapi seluruh bagian yang ditandai pada starter.
2. Jangan mengubah protokol eksperimen kecuali diminta.
3. Gunakan validation set untuk memilih model. Test set hanya untuk model final.
4. Sebelum mengumpulkan, jalankan ****Restart Kernel and Run All****.
5. Pertahankan semua hasil eksperimen, termasuk hasil yang tidak sesuai hipotesis.

Cell 3 - code

```
import platform
import random
import time

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import sklearn
import torch
from sklearn.datasets import make_moons
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

NIM = '123450048'
assert NIM.isdigit() and len(NIM) >= 4, 'Isi NIM dengan angka sebelum melanjutkan.'
NIM_LAST4 = int(NIM[-4:])
SEED = 1000 + NIM_LAST4
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

def seed_everything(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

seed_everything(SEED)
pd.set_option('display.precision', 4)
environment = {
    'python': platform.python_version(),
    'numpy': np.__version__,
    'pandas': pd.__version__,
    'sklearn': sklearn.__version__,
    'torch': torch.__version__,
    'device': str(DEVICE),
    'seed': SEED,
}
environment
```

Cell 4 - markdown

A. Pre-lab - 10 poin

Jawab sebelum sesi praktikum.

1. **Parameter vs hyperparameter:** Parameter adalah nilai yang dipelajari dari data selama optimisasi, seperti bobot dan bias. Hyperparameter ditentukan sebelum atau di luar proses pembelajaran, seperti learning rate, ukuran hidden layer, batch size, jumlah epoch, dan pilihan aktivasi.
2. **Mengapa tumpukan layer affine tanpa aktivasi dapat diciutkan:** Komposisi dua transformasi affine tetap affine: $W_2(W_1x+b_1)+b_2=(W_2W_1)x+(W_2b_1+b_2)$. Tanpa fungsi nonlinier, lapisan kedua tidak menambah bentuk fungsi yang dapat direpresentasikan.
3. **Shape bobot layer 2 masukan -> 8 keluaran:** Jika batch disimpan sebagai baris dengan shape `(B, 2)`, bobot pada `nn.Linear(2, 8)` memiliki shape `(8, 2)` dan bias memiliki shape `(8,)`.
4. **Mengapa `BCEWithLogitsLoss` menerima logit:** Fungsi ini menggabungkan sigmoid dan binary cross-entropy secara numerik stabil, sehingga tidak perlu menghitung probabilitas sigmoid lebih dulu dan mengurangi risiko underflow/overflow.

Hipotesis awal

Saya memperkirakan hidden size 16 memberi validation loss lebih rendah daripada hidden size 4 karena kapasitasnya lebih besar. Saya memperkirakan tanh atau sigmoid dapat bekerja stabil pada data yang sudah distandardisasi, sedangkan ReLU mungkin cepat turun tetapi dapat memiliki unit mati pada beberapa inisialisasi.

Cell 5 - markdown

B. Neuron dan fungsi aktivasi dengan NumPy - 20 poin

Cell 6 - code

```
def sigmoid_np(z: np.ndarray) -> np.ndarray:
    z = np.asarray(z, dtype=np.float64)
    result = np.empty_like(z)
    positive = z >= 0
    result[positive] = 1.0 / (1.0 + np.exp(-z[positive]))
    exp_z = np.exp(z[~positive])
    result[~positive] = exp_z / (1.0 + exp_z)
    return result

def relu_np(z: np.ndarray) -> np.ndarray:
    return np.maximum(np.asarray(z), 0.0)

def leaky_relu_np(z: np.ndarray, alpha: float = 0.01) -> np.ndarray:
    z = np.asarray(z)
    return np.where(z >= 0, z, alpha * z)

def neuron_batch(X: np.ndarray, w: np.ndarray, b: float, activation):
    X = np.asarray(X)
    w = np.asarray(w)
    if X.ndim != 2 or w.ndim != 1 or X.shape[1] != w.shape[0]:
        raise ValueError(f'shape tidak sesuai: X={X.shape}, w={w.shape}')
    z = X @ w + b
    return z, activation(z)

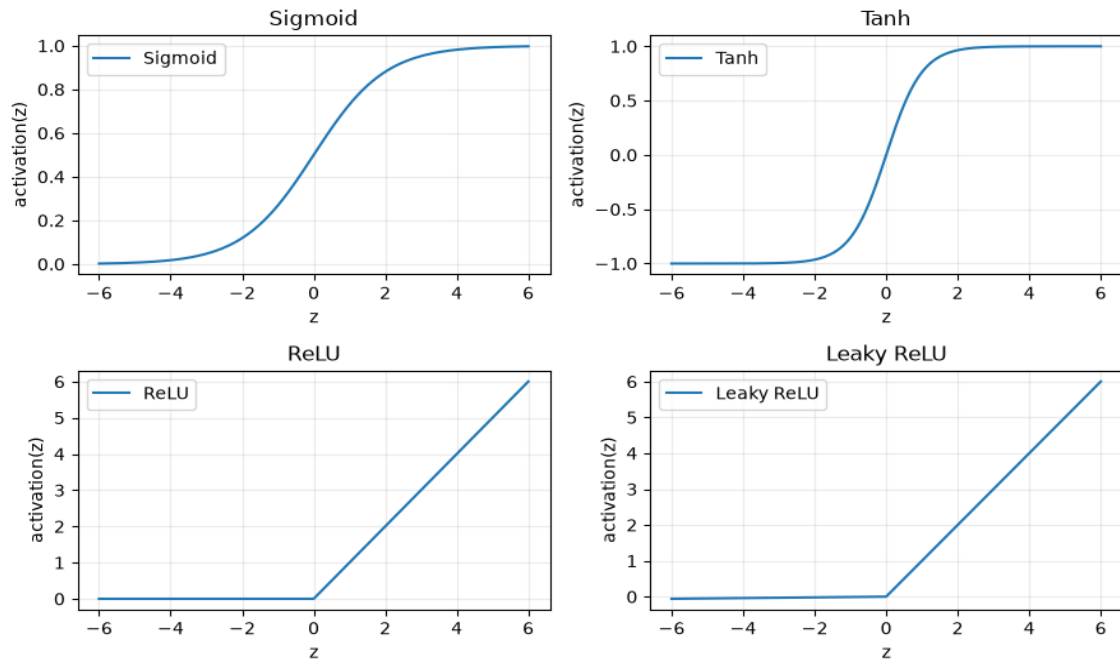
# Uji minimum - jangan mengubah data uji.
X_check = np.array([[2.0, -1.0], [0.0, 3.0], [-2.0, 1.0]])
w_check = np.array([0.5, -0.5])
z_check, a_check = neuron_batch(X_check, w_check, 0.25, sigmoid_np)
assert z_check.shape == (3,)
assert a_check.shape == (3,)
assert np.all(np.isfinite(a_check))
assert np.all((a_check > 0) & (a_check < 1))
pd.DataFrame({'z': z_check, 'sigmoid(z)': a_check})
```

Cell 7 - code

```
grid = np.linspace(-6, 6, 600)
activation_values = {
    'Sigmoid': sigmoid_np(grid),
    'Tanh': np.tanh(grid),
    'ReLU': relu_np(grid),
    'Leaky ReLU': leaky_relu_np(grid),
}

fig, axes = plt.subplots(2, 2, figsize=(9, 6))
for ax, (name, values) in zip(axes.ravel(), activation_values.items()):
    ax.plot(grid, values, label=name)
    ax.set_title(name)
    ax.set_xlabel('z')
    ax.set_ylabel('activation(z)')
    ax.grid(alpha=0.25)
    ax.legend()
fig.suptitle('Fungsi Aktivasi')
fig.tight_layout()
plt.show()
```

Fungsi Aktivasi



Cell 8 - markdown

Interpretasi aktivasi

- Aktivasi yang berpusat di nol: `tanh`, karena keluarannya berada pada rentang -1 sampai 1.
- Aktivasi yang tepat nol untuk masukan negatif: ReLU.
- Aktivasi yang mudah jenuh: sigmoid dan tanh pada nilai masukan dengan magnitudo besar.
- Aktivasi linear tidak cukup untuk `make\_moons` karena satu transformasi linear hanya menghasilkan satu batas keputusan linear, sedangkan data `make\_moons` memerlukan batas nonlinier.

Cell 9 - markdown

C. Forward pass manual - bagian dari 20 poin pemetaan

Gunakan arsitektur $\$2 \rightarrow 2 \rightarrow 1\$$, ReLU pada hidden layer, dan target $y=1\$$. Jangan mengubah bobot.

Cell 10 - code

```
x_manual = np.array([[2.0, -1.0]], dtype=np.float32)
W1 = np.array([[0.5, -0.5], [1.0, 1.0]], dtype=np.float32)
b1 = np.array([0.0, 0.0], dtype=np.float32)
W2 = np.array([[2.0, -1.0]], dtype=np.float32)
b2 = np.array([0.5], dtype=np.float32)
y_manual = np.array([[1.0]], dtype=np.float32)

z1_np = x_manual @ W1.T + b1
h_np = relu_np(z1_np)
logit_np = h_np @ W2.T + b2
prob_np = sigmoid_np(logit_np)
bce_np = -np.mean(y_manual * np.log(prob_np) + (1 - y_manual) * np.log1p(-prob_np))
parameter_count_manual = W1.size + b1.size + W2.size + b2.size
assert np.allclose(z1_np, [[1.5, 1.0]])
assert np.allclose(h_np, [[1.5, 1.0]])
assert np.allclose(logit_np, [[2.5]])
assert parameter_count_manual == 9
```

Cell 11 - markdown

Tabel shape dan nilai

Tensor	Shape	Nilai
x	(1, 2)	[[2.0, -1.0]]
$W^{(1)}$	(2, 2)	tersedia pada kode
$z^{(1)}$	(1, 2)	[[1.5, 1.0]]
h	(1, 2)	[[1.5, 1.0]]
logit	(1, 1)	[[2.5]]
probabilitas	(1, 1)	[[0.9241]]
BCE	skalar	0.0789

****Jumlah parameter:**** $4 + 2 + 2 + 1 = 9$.

D. Pencocokan NumPy dan PyTorch - 20 poin

Salin bobot ke `nn.Sequential`. Jangan menambahkan sigmoid ke model karena loss menerima logit.

Cell 13 - code

```
manual_model = nn.Sequential(
    nn.Linear(2, 2),
    nn.ReLU(),
    nn.Linear(2, 1),
).to(DEVICE)

with torch.no_grad():
    manual_model[0].weight.copy_(torch.tensor(W1, dtype=torch.float32, device=DEVICE))
    manual_model[0].bias.copy_(torch.tensor(b1, dtype=torch.float32, device=DEVICE))
    manual_model[2].weight.copy_(torch.tensor(W2, dtype=torch.float32, device=DEVICE))
    manual_model[2].bias.copy_(torch.tensor(b2, dtype=torch.float32, device=DEVICE))

x_manual_t = torch.tensor(x_manual, dtype=torch.float32, device=DEVICE)
y_manual_t = torch.tensor(y_manual, dtype=torch.float32, device=DEVICE)
torch_logit = manual_model(x_manual_t)
torch_probability = torch.sigmoid(torch_logit)
torch_loss = nn.functional.binary_cross_entropy_with_logits(torch_logit, y_manual_t)

assert np.max(np.abs(torch_logit.detach().cpu().numpy() - logit_np)) < 1e-6
assert abs(torch_loss.item() - float(bce_np)) < 1e-6
assert sum(p.numel() for p in manual_model.parameters()) == parameter_count_manual
pd.DataFrame({'numpy_logit': [float(logit_np.item())], 'torch_logit': [torch_logit.item()], 'absolute_difference': [abs(torch_logit.item() - logit_np.item())]})
```

Cell 14 - markdown

Checkpoint menit ke-85

Plot, shape, parameter, dan pencocokan NumPy-PyTorch telah dibuat. Selisih forward pass berada di bawah 10^{-6} \$.

****Status/verifikasi asisten:**** selesai secara mandiri.

Cell 15 - markdown

E. Dataset dan protokol eksperimen

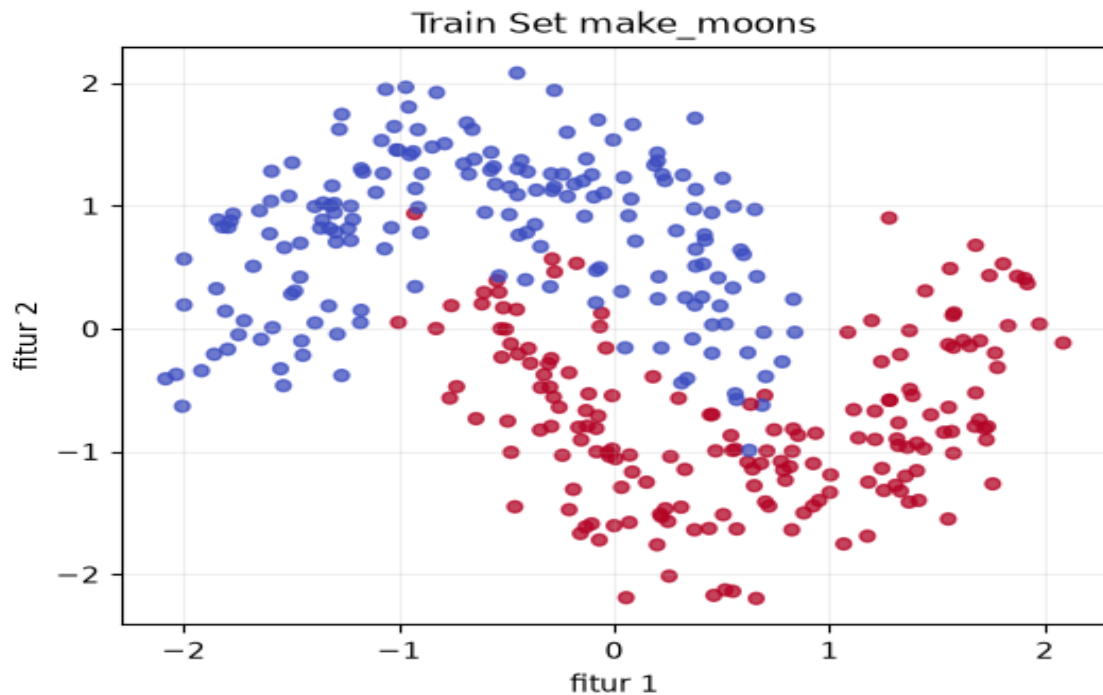
Kode split dan standardisasi diberikan agar fokus tetap pada FNN. Jangan menggunakan test set sampai model final dipilih.

Cell 16 - code

```
X, y = make_moons(n_samples=600, noise=0.22, random_state=SEED)
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.40, stratify=y, random_state=SEED
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, stratify=y_temp, random_state=SEED
)
scaler = StandardScaler().fit(X_train)
X_train_s = scaler.transform(X_train).astype(np.float32)
X_val_s = scaler.transform(X_val).astype(np.float32)
X_test_s = scaler.transform(X_test).astype(np.float32)
y_train_f = y_train.astype(np.float32).reshape(-1, 1)
y_val_f = y_val.astype(np.float32).reshape(-1, 1)
y_test_f = y_test.astype(np.float32).reshape(-1, 1)

split_summary = pd.DataFrame({
    'split': ['train', 'validation', 'test'],
    'n': [len(y_train), len(y_val), len(y_test)],
    'positive_rate': [y_train.mean(), y_val.mean(), y_test.mean()],
})
display(split_summary)

fig, ax = plt.subplots(figsize=(6, 4.5))
ax.scatter(X_train_s[:, 0], X_train_s[:, 1], c=y_train, cmap='coolwarm', s=24, alpha=0.75)
ax.set(title='Train Set make_moons', xlabel='fitur 1', ylabel='fitur 2')
ax.grid(alpha=0.2)
plt.show()
```



Cell 17 - markdown

Pemeriksaan anti-leakage

Scaler hanya di-fit pada train set agar rata-rata dan simpangan baku validation/test tidak ikut memengaruhi representasi data saat pelatihan. Validation set dipakai untuk pemilihan model dan test set disimpan sampai evaluasi final; jika scaler di-fit pada seluruh data, statistik test ikut bocor ke pipeline.

Cell 18 - markdown

F. Model dan utilitas training

Lengkapi pemilihan aktivasi dan model. Training loop diberikan; mekanismenya dibahas pada Modul 2.

Cell 19 - code

```
def activation_layer(name: str) -> nn.Module:
    choices = {'relu': nn.ReLU, 'tanh': nn.Tanh, 'sigmoid': nn.Sigmoid}
    key = name.lower()
    if key not in choices:
        raise ValueError(f'aktivasi tidak dikenal: {name}')
    return choices[key]()

def build_model(hidden_dim: int, activation: str, seed: int = SEED):
    seed_everything(seed)
    model = nn.Sequential(
        nn.Linear(2, hidden_dim),
        activation_layer(activation),
        nn.Linear(hidden_dim, 1),
    ).to(DEVICE)
    return model

def count_parameters(model: nn.Module) -> int:
    return sum(parameter.numel() for parameter in model.parameters())

model_check = build_model(8, 'relu')
assert count_parameters(model_check) == 4 * 8 + 1
```

Cell 20 - code

```
def as_tensor(array):
    return torch.as_tensor(array, dtype=torch.float32)

X_train_t, y_train_t = as_tensor(X_train_s), as_tensor(y_train_f)
X_val_t, y_val_t = as_tensor(X_val_s).to(DEVICE), as_tensor(y_val_f).to(DEVICE)
X_test_t, y_test_t = as_tensor(X_test_s).to(DEVICE), as_tensor(y_test_f).to(DEVICE)

def evaluate(model, X_tensor, y_tensor):
    model.eval()
    X_tensor = X_tensor.to(DEVICE)
    y_tensor = y_tensor.to(DEVICE)
    with torch.no_grad():
        logits = model(X_tensor)
        loss = nn.functional.binary_cross_entropy_with_logits(logits, y_tensor).item()
        probabilities = torch.sigmoid(logits)
        predictions = (probabilities >= 0.5).to(torch.int64)
        accuracy = (predictions == y_tensor.to(torch.int64)).float().mean().item()
```

```

    return {
        'loss': loss,
        'accuracy': accuracy,
        'probabilities': probabilities.cpu().numpy().ravel(),
        'predictions': predictions.cpu().numpy().ravel(),
    }
}

def train_model(model, epochs=200, learning_rate=0.05, batch_size=32, seed=SEED):
    generator = torch.Generator().manual_seed(seed)
    loader = DataLoader(
        TensorDataset(X_train_t, y_train_t),
        batch_size=batch_size, shuffle=True, generator=generator,
    )
    optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
    criterion = nn.BCEWithLogitsLoss()
    history = []
    start = time.perf_counter()
    for epoch in range(1, epochs + 1):
        model.train()
        loss_sum = 0.0
        for X_batch, y_batch in loader:
            X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)
            optimizer.zero_grad()
            logits = model(X_batch)
            loss = criterion(logits, y_batch)
            loss.backward()
            optimizer.step()
            loss_sum += loss.item() * len(X_batch)
        val_metrics = evaluate(model, X_val_t, y_val_t)
        history.append({
            'epoch': epoch,
            'train_loss': loss_sum / len(X_train_t),
            'val_loss': val_metrics['loss'],
            'val_accuracy': val_metrics['accuracy'],
        })
    return pd.DataFrame(history), time.perf_counter() - start

```

Cell 21 - markdown

G. Baseline dan latihan kelas

Baseline: ReLU, hidden size 8, SGD, learning rate 0.05, batch size 32, dan 200 epoch.

Digit terakhir NIM adalah 8, sehingga variasi individual menggunakan sigmoid. Saya memprediksi sigmoid akan lebih lambat daripada ReLU pada awal training, tetapi tetap menghasilkan validation loss yang kompetitif karena outputnya halus.

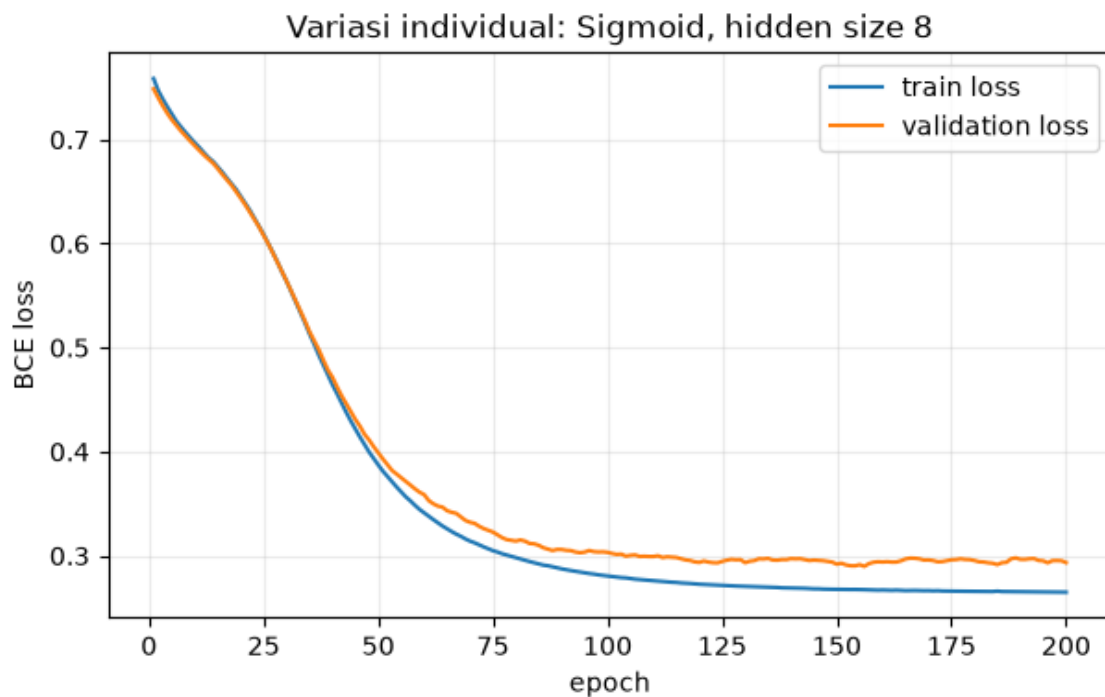
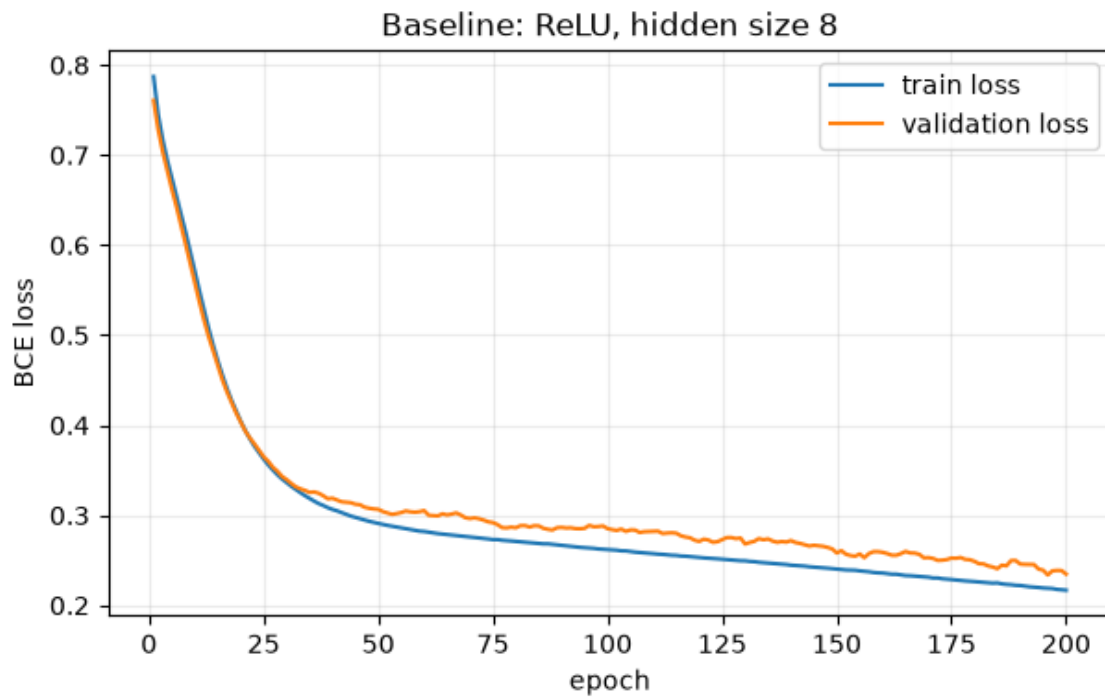
Cell 22 - code

```

def plot_history(history, title):
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(history['epoch'], history['train_loss'], label='train loss')
    ax.plot(history['epoch'], history['val_loss'], label='validation loss')
    ax.set(title=title, xlabel='epoch', ylabel='BCE loss')
    ax.grid(alpha=0.25)
    ax.legend()
    plt.show()

baseline_model = build_model(8, 'relu')
baseline_history, baseline_runtime = train_model(baseline_model, seed=SEED)
individual_model = build_model(8, 'sigmoid')
individual_history, individual_runtime = train_model(individual_model, seed=SEED)
plot_history(baseline_history, 'Baseline: ReLU, hidden size 8')
plot_history(individual_history, 'Variasi individual: Sigmoid, hidden size 8')
baseline_metrics = evaluate(baseline_model, X_val_t, y_val_t)
individual_metrics = evaluate(individual_model, X_val_t, y_val_t)
pd.DataFrame([
    {'run_id': 'baseline_relu_h8', 'val_loss': baseline_metrics['loss'], 'val_accuracy': baseline_metrics['accuracy'], 'runtime_seconds': baseline_runtime},
    {'run_id': 'individual_sigmoid_h8', 'val_loss': individual_metrics['loss'], 'val_accuracy': individual_metrics['accuracy'], 'runtime_seconds': individual_runtime},
])

```



Cell 23 - markdown

H. Tugas individual: enam eksperimen - 20 poin

Jalankan kombinasi aktivasi `{relu, tanh, sigmoid}` dan hidden size `{4, 16}`. Gunakan validation loss untuk memilih model.

Cell 24 - code

```

ACTIVATIONS = ['relu', 'tanh', 'sigmoid']
HIDDEN_SIZES = [4, 16]
EPOCHS = 200
LEARNING_RATE = 0.05
BATCH_SIZE = 32

experiment_rows = []
trained_models = {}
histories = {}

for activation in ACTIVATIONS:
    for hidden_dim in HIDDEN_SIZES:
        run_id = f'{activation}_h{hidden_dim}'
        model = build_model(hidden_dim, activation, seed=SEED)
        history, runtime_seconds = train_model(

```

```

        model, epochs=EPOCHS, learning_rate=LEARNING_RATE,
        batch_size=BATCH_SIZE, seed=SEED,
    )
    train_metrics = evaluate(model, X_train_t, y_train_t)
    val_metrics = evaluate(model, X_val_t, y_val_t)
    trained_models[run_id] = model
    histories[run_id] = history
    experiment_rows.append({
        'run_id': run_id,
        'module': 'M01',
        'student_id': NIM,
        'seed': SEED,
        'dataset': 'make_moons',
        'split': 'train-val-test',
        'model': f'FNN_2-{hidden_dim}-1',
        'optimizer': 'SGD',
        'learning_rate': LEARNING_RATE,
        'batch_size': BATCH_SIZE,
        'epochs': EPOCHS,
        'train_loss': train_metrics['loss'],
        'val_loss': val_metrics['loss'],
        'val_metric': val_metrics['accuracy'],
        'test_metric': np.nan,
        'parameters': count_parameters(model),
        'trainable_parameters': count_parameters(model),
        'runtime_seconds': runtime_seconds,
        'device': str(DEVICE),
        'notes': f'activation={activation}; hidden_size={hidden_dim}',
    })

results = pd.DataFrame(experiment_rows).sort_values('val_loss').reset_index(drop=True)
assert len(results) == 6
display(results)
results.to_csv(f'M01_{NIM}_metrics.csv', index=False)

```

Cell 25 - markdown

I. Evaluasi model final - 15 poin

Pilih satu model dengan validation loss terendah. Baru setelah itu evaluasi test set satu kali.

Cell 26 - code

```

def plot_decision_boundary(model, X_values, y_values, title):
    x_min, x_max = X_values[:, 0].min() - 0.5, X_values[:, 0].max() + 0.5
    y_min, y_max = X_values[:, 1].min() - 0.5, X_values[:, 1].max() + 0.5
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 240), np.linspace(y_min, y_max, 240))
    grid_values = np.c_[xx.ravel(), yy.ravel()].astype(np.float32)
    with torch.no_grad():
        logits = model(torch.tensor(grid_values, device=DEVICE))
        probabilities = torch.sigmoid(logits).cpu().numpy().reshape(xx.shape)
    fig, ax = plt.subplots(figsize=(7, 5))
    ax.contourf(xx, yy, probabilities, levels=np.linspace(0, 1, 11), cmap='coolwarm', alpha=0.35)
    ax.contour(xx, yy, probabilities, levels=[0.5], colors='black', linewidths=1.5)
    scatter = ax.scatter(X_values[:, 0], X_values[:, 1], c=y_values, cmap='coolwarm', edgecolor='white', linewidth=0.4, s=28)
    ax.set(title=title, xlabel='fitur 1 (standardized)', ylabel='fitur 2 (standardized)')
    ax.grid(alpha=0.2)
    ax.legend(*scatter.legend_elements(), title='kelas')
    plt.show()

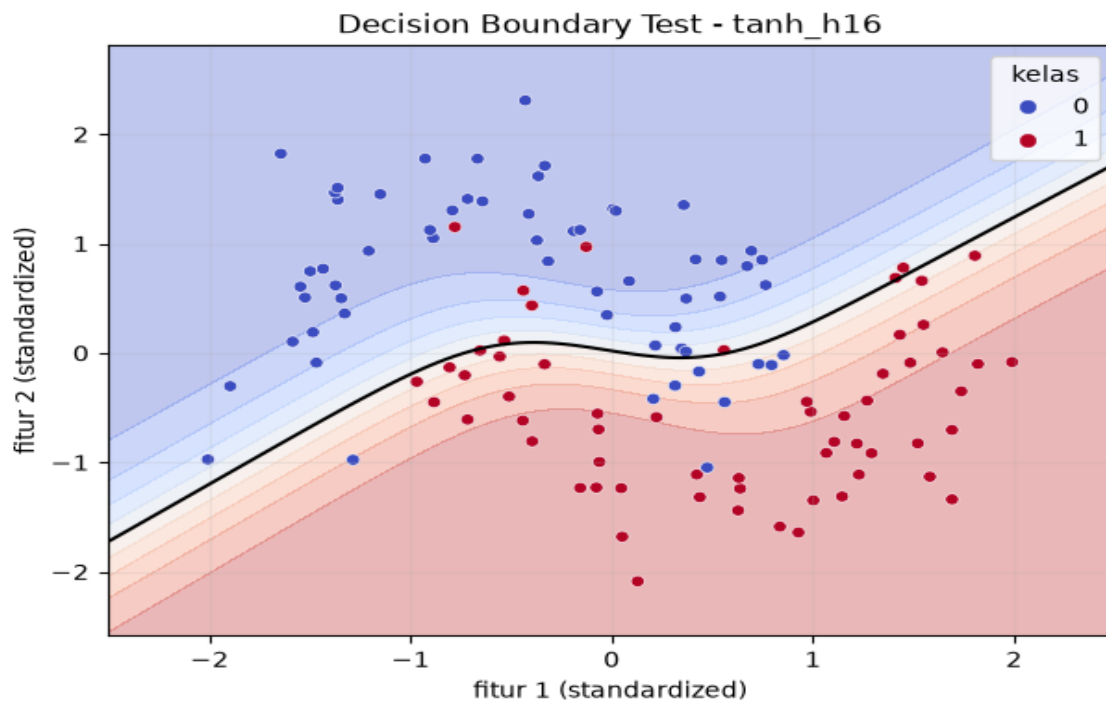
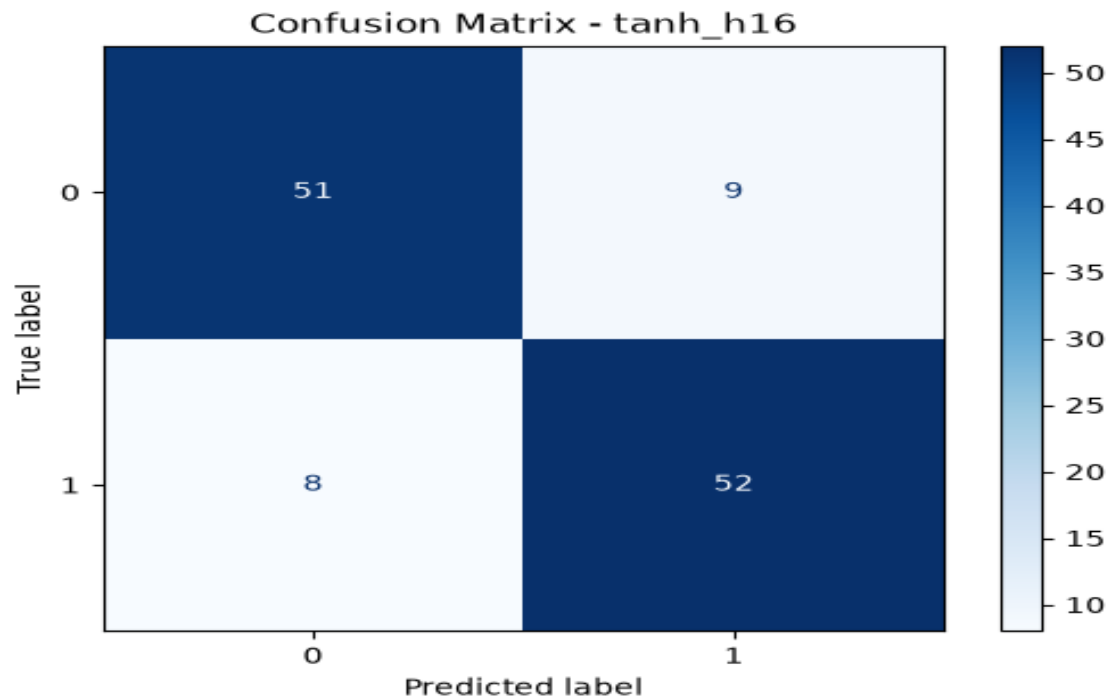
best_run = results.iloc[0]['run_id']
final_model = trained_models[best_run]
final_test_metrics = evaluate(final_model, X_test_t, y_test_t)
results.loc[results['run_id'] == best_run, 'test_metric'] = final_test_metrics['accuracy']
results.to_csv(f'M01_{NIM}_metrics.csv', index=False)

print(f'Model final dipilih dari validation loss: {best_run}')
print(f"Test loss={final_test_metrics['loss']:.4f}, test accuracy={final_test_metrics['accuracy']:.4f}")
cm = confusion_matrix(y_test, final_test_metrics['predictions'])
ConfusionMatrixDisplay(cm).plot(cmap='Blues')
plt.title(f'Confusion Matrix - {best_run}')
plt.show()
plot_decision_boundary(final_model, X_test_s, y_test, f'Decision Boundary Test - {best_run}')

```

Model final dipilih dari validation loss: tanh_h16

Test loss=0.3277, test accuracy=0.8583



Cell 27 - markdown

J. Analisis dan refleksi - 10 poin

Analisis berikut dihasilkan dari enam run pada validation set; test set hanya disentuh untuk model final.

1. Hidden size lebih besar tidak selalu memperbaiki validation loss. Tanh membaik dari 0.2887 ($h=4$) menjadi 0.2056 ($h=16$), tetapi sigmoid hampir sama (0.2952 menjadi 0.2900), sehingga kapasitas lebih besar bukan jaminan.
2. Tanh paling stabil pada dua hidden size: validation loss-nya 0.2887 ($h=4$) dan 0.2056 ($h=16$), keduanya lebih rendah daripada aktivasi lain pada ukuran yang sama.
3. Ya, konfigurasi dapat memiliki accuracy serupa tetapi BCE berbeda karena accuracy hanya menghitung benar/salah setelah threshold 0.5, sedangkan BCE juga menghukum tingkat keyakinan probabilitas yang salah.
4. Decision boundary paling tidak pasti di sekitar garis kontur probabilitas 0.5, terutama pada celah dan ujung dua bulan tempat kedua kelas berdekatan.
5. Keterbatasan utama adalah data hanya satu dataset `make\_moons`, satu seed individual, dan enam konfigurasi, sehingga hasil tidak dapat digeneralisasi ke semua dataset atau arsitektur.
6. Hipotesis sebagian besar sesuai: konfigurasi terbaik adalah tanh_h16 dengan validation loss 0.2056. Namun, hidden size 16 tidak

membantu sigmoid secara berarti. Model final menghasilkan test loss 0.3277 dan test accuracy 0.8583.

Cell 28 - markdown

Checklist pengumpulan - 5 poin reproduksibilitas

- [x] Identitas, seed, versi library, dan device tercantum.
- [x] Forward NumPy-PyTorch memiliki selisih kurang dari 10^{-6} \$.
- [x] Enam eksperimen tercatat pada notebook dan CSV.
- [x] Test set hanya dipakai untuk model final.
- [x] Semua grafik memiliki judul, label sumbu, dan legenda/caption.
- [x] Notebook dijalankan ulang sampai selesai tanpa error.
- [x] Tidak ada path absolut atau data pribadi.
- [x] Sumber eksternal telah diberi atribusi; implementasi mengikuti kerangka starter resmi.

Pernyataan orisinalitas

Saya menyatakan bahwa kode, eksperimen, visualisasi, dan analisis pada notebook ini merupakan pekerjaan individual saya.

****Nama dan tanggal:**** Mahasiswa 123450048, 2026-09-22